EXPLORAÇÃO DINÂMICA DE AMBIENTES ACADÊMICOS UTILIZANDO REALIDADE AUMENTADA: UM GUIA VIRTUAL PARA A FURB

Luan Lavandoski Guarnieri, Dalton Solano dos Reis – Orientador

Curso de Bacharel em Ciência da Computação
Departamento de Sistemas e Computação
Universidade Regional de Blumenau (FURB) – Blumenau, SC – Brasil

lguarnieri@furb.br, dalton@furb.br

Resumo: *Este artigo apresenta o desenvolvimento de dois aplicativos móveis de Realidade Aumentada para auxiliar visitantes na navegação indoor do Bloco S da Universidade Regional de Blumenau (FURB). A solução é composta por um aplicativo Coletor, responsável por registrar pontos de interesse em RA, e um aplicativo de Navegação, que reconstrói esses pontos e orienta o usuário até o destino por meio de esferas virtuais. A implementação utilizou Unity, AR Foundation, ARCore e leitura de QR Code, permitindo criar rotas sem dependência de GPS ou de informações geoespaciais externas, já que todo o processo ocorre apenas com rastreamento visual. Os testes realizados em ambiente real mostraram que a abordagem com coordenadas relativas e calibração espacial funciona de forma consistente, embora apresente limitações naturais do tracking em movimento. Os resultados indicam que a solução é viável como protótipo e pode ser aprimorada para uso em eventos acadêmicos e outros espaços internos.*

Palavras-chave: *Realidade Aumentada. Navegação indoor. Aplicativo móvel. AR Foundation.*

1 INTRODUÇÃO

Nos últimos anos, as tecnologias de Realidade Aumentada (RA) avançaram além do entretenimento e passaram a ser aplicadas em áreas como educação, indústria, saúde e navegação, impulsionadas pela evolução dos dispositivos móveis e pelo acesso a ferramentas de desenvolvimento (SINGH; BANO, 2021). Essa capacidade de sobrepor elementos virtuais ao ambiente físico torna a RA útil em locais onde a orientação espacial não é intuitiva, especialmente em ambientes internos (CAVALCANTE; SILVA, 2021). Na FURB, o Bloco S é um desses casos: um prédio grande, com vários pavimentos e salas distribuídas em diferentes setores, o que costuma gerar dificuldade para visitantes que não conhecem sua estrutura.

A motivação para este trabalho surgiu a partir de eventos como a Interação FURB, nos quais muitos participantes relatam dificuldade para localizar salas e laboratórios dentro do Bloco S. Como o Sistema de Posicionamento Global (Global Positioning System – GPS) não funciona bem em ambientes internos devido à perda de sinal e baixa precisão, a navegação indoor depende de soluções baseadas em visão computacional e reconhecimento espacial (Cavalcante; Silva, 2021). Levando em consideração esses fatos, esse Trabalho de Conclusão de Curso (TCC) desenvolveu um aplicativo simples e acessível, usando RA, que exibisse indicadores virtuais posicionados sobre o ambiente real.

O problema abordado neste trabalho é a ausência de uma ferramenta prática que auxilie visitantes na orientação interna dentro da universidade. Mesmo com sinalizações físicas, quem não conhece o prédio pode se desorientar com facilidade. Com RA, torna-se possível exibir esferas virtuais indicando a direção dos pontos de interesse previamente mapeados. Considerando esse problema, o objetivo geral deste artigo desenvolver e testar um aplicativo de RA capaz de auxiliar visitantes na localização de ambientes específicos da FURB. Os objetivos específicos deste trabalho consistem em identificar e mapear os pontos de interesse do Bloco S da FURB, propor uma solução de navegação indoor baseada em Realidade Aumentada para ambientes acadêmicos, aplicar conceitos de grafos para o cálculo de rotas em ambientes internos e avaliar a usabilidade e a estabilidade da aplicação por meio de testes realizados no Bloco S, considerando limitações relacionadas ao tracking e à ancoragem dos elementos virtuais.

A metodologia envolveu estudo das ferramentas de RA, implementação do aplicativo e testes em ambiente real. Durante os testes, observou-se que a estabilidade da ancoragem depende da movimentação do usuário, das condições de luz e da qualidade do rastreamento, comportamento já documentado em aplicativos baseados em ARCore (Google, 2023). Ainda assim, o funcionamento geral pôde ser validado como apoio inicial à orientação interna, justificando o desenvolvimento e análise apresentados nas seções seguintes.

2 FUNDAMENTAÇÃO TEÓRICA

Nesta seção são apresentados os conceitos essenciais para o desenvolvimento do aplicativo. Primeiro, na subseção 2.1, discute-se a RA, os princípios de *tracking* envolvidos no posicionamento de objetos virtuais, e aborda-se o

AR Foundation como a camada de abstração que permite integrar recursos de ARCore e ARKit na Unity. Já a subseção 2.2 apresenta os recursos da Unity utilizados na implementação. Por fim, a subseção 2.3 apresenta os trabalhos correlatos que abordam navegação indoor e uso de RA como guia visual.

2.1 REALIDADE AUMENTADA, TRACKING E AR FOUNDATION

A Realidade Aumentada (RA) permite a sobreposição de elementos digitais ao ambiente físico em tempo real, exigindo que o sistema conheça a posição e a orientação do dispositivo (LOPES; TEICHRIEB, 2020). De acordo com a documentação da Unity (Unity Technologies, 2023a), esse processo depende de técnicas de visão computacional para identificar padrões no ambiente e calcular a pose da câmera. O *tracking* combina informações capturadas pela câmera com dados de sensores inerciais, permitindo estimar movimento e profundidade (GOOGLE, 2023). Plataformas como ARCore e ARKit utilizam essa fusão de sensores para manter objetos virtuais estáveis no espaço (Google, 2023; Apple, 2022). Ainda assim, a precisão do rastreamento varia conforme as condições do ambiente.

Estudos recentes mostram que iluminação inadequada, superfícies com pouca textura e movimentos bruscos da câmera afetam a estabilidade da RA, gerando oscilações na posição dos objetos virtuais (*jitter*) ou desvios acumulados ao longo do tempo (*drift*) (Singh; Bano, 2021). Esses comportamentos são comuns em aplicativos baseados predominantemente em visão computacional e influenciam diretamente a precisão da navegação em aplicações como a desenvolvida neste trabalho.

Já o AR Foundation é o framework oficial da Unity para desenvolvimento de aplicações de Realidade Aumentada multiplataforma. Ele funciona como uma camada de abstração sobre o ARCore e Arquit., permitindo que o mesmo projeto seja executado tanto em Android quanto em iOS sem necessidade de código específico para cada plataforma. De acordo com a documentação oficial da Unity (Unity Technologies, 2023b), o AR Foundation integra recursos de rastreamento de movimento, detecção do ambiente e renderização de objetos virtuais em tempo real. A principal vantagem desse framework para este trabalho foi a possibilidade de trabalhar diretamente com o sistema de *tracking* do ARCore, mantendo a estabilidade da pose da câmera e permitindo ancorar objetos 3D como as esferas que representam os destinos no espaço físico. Além disso, a integração nativa com a Unity facilita o fluxo de desenvolvimento, permitindo criar interfaces, *scripts* e elementos 3D em um mesmo ambiente. Pesquisas recentes apontam o AR Foundation como uma das ferramentas mais utilizadas em contextos acadêmicos devido à sua flexibilidade, documentação extensa e facilidade na prototipagem de aplicações interativas (Nascimento; Filgueiras, 2021; Costa; Amaral, 2022). Esses fatores justificam sua adoção no aplicativo desenvolvido neste trabalho.

2.2 RECURSOS DA UNITY

A Unity é uma das plataformas mais utilizadas para o desenvolvimento de aplicações 3D e de Realidade Aumentada. Essa plataforma engloba um motor de jogos, que oferece suporte nativo para a linguagem C-Sharp, recursos de renderização em tempo real e integração direta com o AR Foundation, o que permite utilizar os sistemas de *tracking* do ARCore e do Arquit. no mesmo projeto (Unity Technologies, 2023a). Esses fatores tornam a Unity adequada tanto para prototipação quanto para desenvolvimento de aplicações móveis completas. A Unity é amplamente utilizada no desenvolvimento de aplicações interativas por permitir a organização de cenas de forma modular, a criação e manipulação de objetos tridimensionais e o controle do comportamento da aplicação por meio de scripts. Além disso, o uso de pretas facilita a reutilização de componentes e a padronização de elementos visuais e funcionais em aplicações baseadas em Realidade Aumentada (Lopes; Teichrieb, 2020; Nascimento; Filgueiras, 2021).

Pesquisas nacionais apontam a Unity como uma ferramenta eficiente para aplicações de RA devido à sua flexibilidade, extensa documentação e grande volume de materiais de apoio para projetos acadêmicos (Lopes; Teichrieb, 2020). Estudos recentes também destacam seu uso em aplicativos de visualização 3D e em aplicações voltadas à orientação espacial, incluindo projetos de navegação *indoor* que utilizam elementos virtuais simples para guiar usuários em ambientes físicos (Santos; Pereira, 2021; Cavalcante; Silva, 2022).

Além do AR Foundation, o Unity permite utilizar bibliotecas externas, como por exemplo a biblioteca de código aberto ZXing. Nesse projeto, a biblioteca ZXing foi utilizada para identificar a posição inicial dentro do Bloco S (da FURB) por meio da leitura de QR Codes. De acordo com a documentação oficial do ZXing Project (Zxing Project, 2023), essa biblioteca é amplamente utilizada em aplicações móveis por ser leve, multiplataforma e capaz de operar mesmo sem conexão com a internet. A decodificação é realizada diretamente sobre os frames da câmera, permitindo extrair o conteúdo do QR Code em tempo real.

A opção por QR Codes se mostrou adequada para este trabalho porque permite identificar ambientes de forma simples e com baixo custo. Esse tipo de marcador como uma alternativa prática para navegação e reconhecimento de locais em dispositivos móveis, especialmente em ambientes educacionais e de orientação espacial (Souza; Lima, 2021). Outros estudos também indicam que o uso de marcadores visuais facilita a integração entre o ambiente físico e a aplicação, servindo como ponto de referência confiável para aplicativos de localização interna (Ferreira; Gomes, 2020).

2.3 TRABALHOS CORRELATOS

Os estudos de Brasil Neto *et al*. (2023), Rocha (2021) e Lu *et al*. (2021) apresentam diferentes abordagens para o uso da Realidade Aumentada na orientação de usuários em ambientes reais. Esses trabalhos foram selecionados por explorarem aplicações que se aproximam dos objetivos deste artigo, especialmente em cenários de navegação, apoio ao usuário e visualização de informações sobre o ambiente. No Quadro 1, é apresentado o estudo de Brasil Neto *et al*., voltado ao turismo inteligente com uso de painéis virtuais. O Quadro 2 resume o trabalho de Rocha (2021), que desenvolveu uma solução de navegação interna e externa para a UFOP utilizando ARKit. Já o Quadro 3 apresenta o aplicativo de Lu *et al*. (2021), que integra ARCore e Unity para navegação híbrida com mapas tridimensionais. Esses três estudos fornecem um panorama importante sobre diferentes formas de aplicar RA em contextos reais, apontando benefícios, limitações e níveis distintos de complexidade técnica. A seguir, cada trabalho é analisado individualmente, destacando suas principais funcionalidades, seus resultados e sua relação com a solução proposta para o Bloco S da FURB.

O trabalho de Brasil Neto *et al*. (2023). propõe um aplicativo móvel de RA voltado ao turismo inteligente, com integração à plataforma GnomonGo (Quadro 1). A solução utiliza ARCore e ARKit para posicionar painéis virtuais no ambiente real, permitindo ao visitante acessar informações complementares como horários, curiosidades e detalhes históricos dos locais visitados. Um ponto relevante do estudo é o uso de elementos gráficos ricos e coloridos, que buscam aumentar a imersão e tornar a experiência mais atrativa.

Quadro 1 – Realidade aumentada no contexto do turismo inteligente

Referência	Brasil Neto *et al*. (2023)
Objetivos	Desenvolver um aplicativo móvel de RA com o intuito de melhorar a experiência do usuário no turismo.
Principais funcionalidades	Sobreposição de painéis virtuais em ambientes reais, descrições de horários de funcionamento e informações adicionais sobre os locais explorados.
Ferramentas de desenvolvimento	Desenvolvido com Flutter, Google ARCore e Apple ARKit, Blender, Firebase Cloud Firestone, Figma e Lucidchart.
Resultados e conclusões	As avaliações de funcionalidade foram bem positivas, porém identificados alguns pontos para melhorias, como a interação e usabilidade. Pois moradores locais tiveram dificuldades de entender a funcionalidade dos cartões interativos sugerindo ter um tutorial e melhorias para menos poluição de tela. E ainda, questões de compatibilidade pois o aplicativo não é suportado em dispositivos que não possuíam giroscópio e bússola. E por fim, o aplicativo foi integrado ao GnomonGo.

Fonte: elaborado pelo autor, 2025.

Durante a avaliação, Brasil Neto *et al*. (2023) observaram boa aceitação inicial, mas também apontaram dificuldades importantes. Moradores locais tiveram problemas para compreender a interação com os cartões virtuais e sugeriram a inclusão de tutoriais para orientar o uso. Além disso, a solução apresentou limitações quanto à compatibilidade com alguns dispositivos, especialmente aqueles que não possuem giroscópio ou bússola. Outro ponto sensível foi a integração com Cloud Anchors, que dificultou a estabilidade da navegação *indoor*.

Esse estudo dialoga com o presente trabalho porque ambos utilizam RA para indicar pontos de interesse. No entanto, o aplicativo de Brasil Neto *et al*. (2023) incorpora recursos que dependem de sensores adicionais e uma interface mais carregada, enquanto a solução desenvolvida neste artigo utiliza apenas os sensores básicos exigidos pelo ARCore e uma interface mais simples, baseada em QR Codes e esferas virtuais.

O estudo de Rocha (2021) apresenta uma solução de navegação interna e externa desenvolvida para a UFOP, utilizando o ARKit em dispositivos iOS (Quadro 2). O aplicativo exibe marcadores virtuais sobre o ambiente real e utiliza uma Application Programming Interface (API) própria para fornecer instruções de navegação. O trabalho de Rocha (2021) destaca o cuidado com a interface, desenvolvida inicialmente no Adobe XD, e a integração com serviços externos para melhorar o fluxo de informações entre o servidor e o aplicativo.

Quadro 2 – Desenvolvimento de um aplicativo para navegação indoor com utilização de realidade aumentada

Referência	Rocha (2021)
Objetivos	Desenvolver uma aplicação para dispositivos iOS que auxilie na locomoção interna e externa na Universidade Federal de Ouro Preto (UFOP) utilizando RA.
Principais funcionalidades	Navegação interna e externa, interface amigável, API de navegação, sistema de marcadores, feedback em tempo real.
Ferramentas de desenvolvimento	Linguagem Swift, Framework ARKit, TypeScript para a API, plataforma Heroku, Adobe XD e bibliotecas auxiliares para leitura de QRCode e Geolocalização.
Resultados e conclusões	Atende bem em quesito usabilidade e estética, mas precisa melhorar em padrões, prevenções de erros, ajuda e documentação. Apresenta um ótimo tempo de resposta, tendo um desenvolvimento bem-sucedido mais ainda com necessidades de ajustes e mais testes com usuários, pela questão do COVID-19 que limitou a realização dos testes.

Fonte: elaborado pelo autor, 2025.

Os testes realizados por Rocha (2021) indicaram boa usabilidade, tempo de resposta eficiente e precisão satisfatória das direções exibidas. No entanto, duas limitações importantes foram identificadas. A primeira é a dependência exclusiva do ecossistema Apple, o que reduz o alcance do aplicativo e impossibilita seu uso em dispositivos Android. A segunda é o número reduzido de participantes durante a fase de testes, em razão da pandemia de COVID-19, o que impediu uma validação mais ampla.

O trabalho de Rocha (2021) se relaciona com os aplicativos desenvolvido para o Bloco S, especialmente no uso da Realidade Aumentada como ferramenta de orientação em ambiente universitário. No entanto, a solução de Rocha (2021) adota uma estrutura mais complexa, com rotas externas e integração de APIs, enquanto o aplicativo proposto neste artigo foi desenvolvido exclusivamente para dispositivos Android, com foco em navegação interna e funcionamento local, sem dependência de servidores externos ou aplicativos adicionais. Embora a arquitetura permita futura adaptação para iOS, essa versão não foi implementada nem testada devido às limitações de tempo e de infraestrutura.

O estudo de Lu *et al*. (2021) apresenta um aplicativo robusto de navegação universitária baseado em ARCore e Unity, aplicado na SUEP, na China (Quadro 3). A proposta integra rotas, mapas tridimensionais e informações virtuais sobre o ambiente físico, oferecendo uma navegação mais detalhada e interativa. Um diferencial do aplicativo de Lu *et al*. (2021) é sua capacidade de sincronizar a posição do usuário em tempo real, combinando dados de diferentes fontes para melhorar a precisão.

Quadro 3 – An ARCore-Based Augmented Reality Campus Navigation System

Referência	Lu *et al.* (2021)
Objetivos	Desenvolver e avaliar um aplicativo de navegação baseado em RA que integre de forma eficaz a navegação interna e externa em um campus universitário melhorando a precisão e a experiência do usuário em comparação com aplicativos tradicionais.
Principais funcionalidades	Navegação interna e externa, planejamento de rotas, sincronização de posição, informações aumentadas, funcionalidade de buscas de livros, experiência do usuário, comparação com aplicativos tradicionais.
Ferramentas de desenvolvimento	Framework ARCode com o motor de jogos Unity.
Resultados e conclusões	O siste aplicativo ma oferece maior precisão e uma experiência ao usuário mais rica comparado a soluções já existentes. A avaliação com os estudantes apresentou uma ótima satisfação como a navegação interna e externa, destacando a eficiência do aplicativo para novos alunos em ambientes complexos como bibliotecas que mostra onde exatamente está o livro.

Fonte: elaborado pelo autor, 2025.

Lu *et al*. (2021) relatam que o aplicativo obteve excelente aceitação dos estudantes, especialmente em ambientes complexos como bibliotecas, onde a aplicação consegue indicar exatamente a localização de livros nas estantes. A qualidade visual dos mapas, aliada à fluidez das rotas, contribuiu para uma experiência mais intuitiva e confiável.

A comparação com o presente trabalho é relevante porque ambos utilizam Realidade Aumentada para orientar usuários em ambientes internos. Entretanto, a solução de Lu *et al*. (2021) apresenta maior complexidade visual e funcional, pois opera com modelos tridimensionais completos e sincronização contínua de posição. Os aplicativos desenvolvidos para o Bloco S seguem uma abordagem mais simples e direta, utilizando apenas indicadores visuais essenciais, sem necessidade de mapas 3D ou infraestrutura adicional, o que torna o aplicativo mais leve e adequado para o contexto específico da navegação interna.

3 DESCRIÇÃO DOS APLICATIVOS

Esta seção apresenta a especificação, implementação e funcionamento dos dois aplicativos desenvolvidos, aqui denominados como Coletor e Navegação. O aplicativo Coletor é utilizado para registrar, identificar e estruturar os pontos de interesse do Bloco S por meio da Realidade Aumentada. Já o aplicativo Navegação realiza a leitura dos QR Codes posicionados no ambiente e exibe as esferas virtuais que indicam os destinos mapeados.

A subseção 3.1 descreve os requisitos de cada aplicativo, a forma como os dados foram organizados e o método utilizado para associar os pontos cadastrados pelo Coletor aos elementos exibidos na Navegação. Já a subseção 3.2 apresenta os detalhes da implementação, incluindo as ferramentas utilizadas, a estrutura interna de cada aplicativo, o fluxo de funcionamento e os principais desafios técnicos enfrentados durante o desenvolvimento. O propósito dessas subseções é fornecer uma visão clara e completa dos aplicativos e das decisões que orientaram os seus desenvolvimentos.

3.1 ESPECIFICAÇÃO

A especificação do aplicativo foi organizada em dois módulos principais, cada um com objetivos claros dentro do fluxo geral de navegação. O aplicativo Coletor é responsável por registrar e estruturar os pontos de interesse do Bloco S diretamente no ambiente real, permitindo ao usuário posicionar esferas no espaço por meio da câmera e associar a cada uma delas informações como nome, bloco e andar. Já o aplicativo de Navegação utiliza os QR Codes distribuídos nos corredores para identificar a posição inicial do usuário e exibir, na câmera, as esferas virtuais correspondentes aos pontos mapeados.

Nesta subseção são apresentados os Requisitos Funcionais (RFs) e Requisitos Não Funcionais (RNFs) de cada módulo, definindo o comportamento esperado do aplicativo e as restrições que orientaram o desenvolvimento. Também é apresentada a estrutura básica dos dados utilizada para armazenar as informações coletadas, como o formato do arquivo Javascript Object Notation (JSON) gerado pelo Coletor e os campos necessários para reconstrução das esferas no aplicativo de Navegação. Os fluxos de funcionamento e os principais casos de uso são apresentados ao longo desta seção, enquanto os diagramas de classes e a estrutura interna dos módulos estão detalhados nos Apêndices A e B.

3.1.1 Requisitos do aplicativo Coletor

O aplicativo Coletor foi desenvolvido para registrar os pontos de interesse no Bloco S utilizando recursos de Realidade Aumentada. O objetivo é permitir que o usuário crie um mapeamento preciso dos ambientes reais, associando informações descritivas (bloco, corredor, sala e descrição) com a posição espacial capturada pela câmera. Todas as informações registradas são posteriormente exportadas em um arquivo JSON, que serve como base para o aplicativo de Navegação reconstruir o grafo de pontos em tempo real.

Para o aplicativo Coletor, foram definidos os seguintes casos de uso principais: cadastrar ponto, capturar posição em Realidade Aumentada, visualizar prévia do ponto, editar ou excluir pontos cadastrados e exportar os dados coletados. Esses casos de uso representam as interações essenciais entre o usuário responsável pelo mapeamento e o sistema, servindo de base para os requisitos funcionais apresentados a seguir.

A seguir são apresentados os RFs (Quadro 4) e RNFs (Quadro 5) definidos para o aplicativo Coletor, que orientaram tanto o fluxo de interação quanto a implementação interna.

Quadro 4 – RFs do Coletor

RF01	O aplicativo deve permitir o usuário cadastrar um novo ponto com nome, bloco e andar.
RF02	O aplicativo deve permitir o usuário capturar a posição tridimensional do ponto utilizando AR Foundation.
RF03	O aplicativo deve permitir o usuário visualizar na cena AR a esfera representando o ponto antes de salvar.
RF04	O aplicativo deve permitir o usuário salvar os pontos cadastrados em uma lista interna.
RF05	O aplicativo deve permitir o usuário editar ou excluir pontos antes da exportação final.
RF06	O aplicativo deve permitir o usuário exportar todos os pontos cadastrados para um arquivo JSON.
RF07	O aplicativo deve impedir o usuário cadastrar de pontos sem nome ou sem posição AR válida.

Fonte: elaborado pelo autor, 2025.

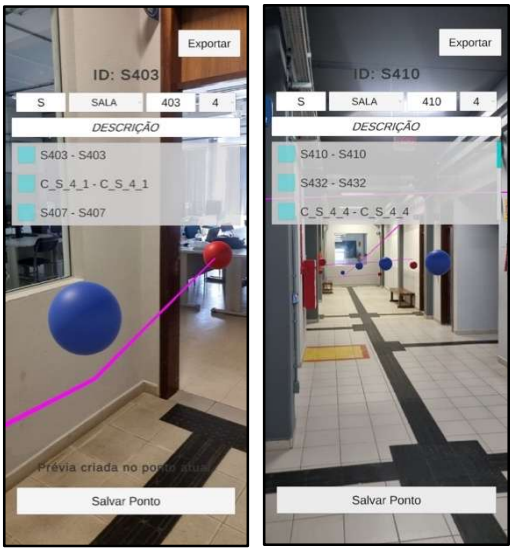
Quadro 5 – RNFs do `Coletor`

RNF01	O aplicativo deve ser compatível com dispositivos Android com suporte a ARCore.
RNF02	O tempo de captura da posição AR não deve exceder dois segundos em condições normais.
RNF03	O JSON gerado deve seguir formato padronizado e legível.
RNF04	O aplicativo deve funcionar offline.

Fonte: elaborado pelo autor, 2025.

Ao iniciar o Coletor (Figura 1a), o usuário visualiza a tela principal do aplicativo Coletor, onde deve preencher os campos bloco, corredor, sala e descrição. Esses campos são obrigatórios, já que compõem a estrutura do arquivo JSON final. O aplicativo Coletor impede o avanço caso algum campo esteja vazio, o que evita inconsistências durante o processo de exportação. Após preencher as informações iniciais, o usuário ativa o modo de captura em Realidade Aumentada. Nessa etapa, a câmera do dispositivo é aberta e o AR Foundation inicia o rastreamento do ambiente. O aplicativo detecta superfícies, ajusta a orientação da cena e procura pontos estáveis onde a esfera virtual poderá ser posicionada. Ao tocar sobre a tela enquanto está no modo AR, o Coletor exibe um preview da esfera no ponto selecionado. Esse preview não é salvo automaticamente, funcionando como uma etapa intermediária para que o usuário avalie se a posição está adequada antes de confirmar o registro definitivo. Esse mecanismo reduz erros de posicionamento e permite correções rápidas (Figura 1b).

Figura 1 – Telas do `Coletor`: (a) inicial; (b) grafo visual das previas

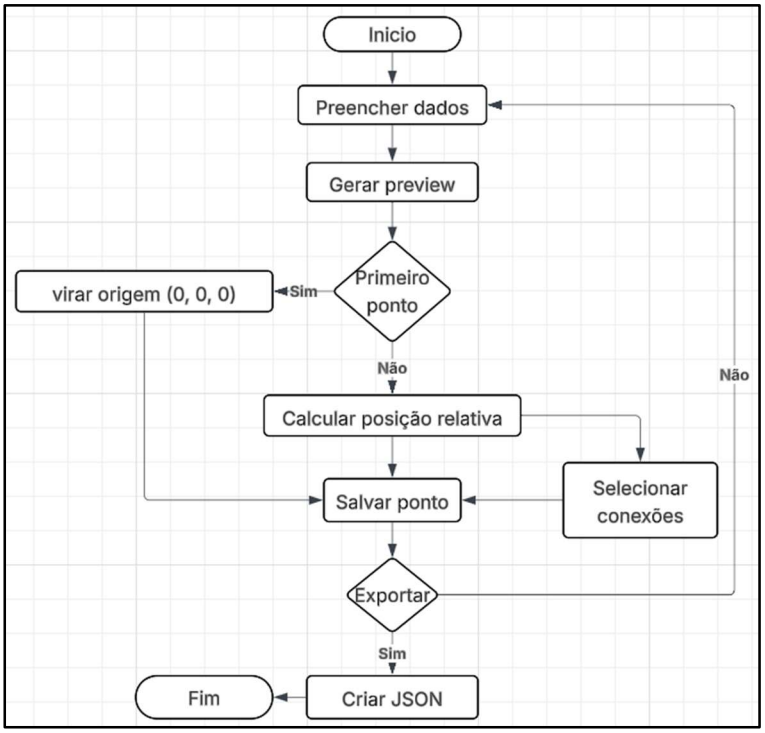


Fonte: elaborado pelo autor, 2025.

O ponto só é registrado quando o usuário pressiona o botão Salvar Ponto. Esse botão faz a captura exata da posição tridimensional (x, y, z) detectada pelo AR Foundation e a vincula às informações textuais inseridas anteriormente. Toques adicionais na tela substituem apenas o preview, sem afetar os pontos já salvos. Dessa forma, o Coletor mantém uma lista interna com todos os pontos registrados durante a sessão. O usuário pode revisar essa lista, excluir pontos que foram criados incorretamente ou editar informações textuais antes da exportação. Esse controle garante que apenas dados válidos e consistentes sejam enviados para o arquivo final.

Quando o mapeamento estiver concluído, o usuário utiliza o botão Exportar para gerar o arquivo JSON contendo todos os pontos cadastrados. O arquivo inclui tanto as informações textuais quanto as coordenadas espaciais capturadas, mantendo uma estrutura padronizada que será interpretada pelo aplicativo de Navegação. Esse fluxo simples e sequencial reduz erros, facilita o processo de coleta em campo e assegura que o conjunto final de pontos seja consistente com o ambiente físico, servindo de base confiável para o funcionamento do aplicativo de navegação. Para facilitar a visualização do processo de coleta e exportação dos pontos, o fluxograma a seguir apresenta o fluxo de funcionamento do aplicativo Coletor (Figura 2).

Figura 2 – Fluxograma do aplicativo coletor



Fonte: elaborado pelo autor, 2025.

3.1.2 Requisitos do aplicativo de Navegação

O aplicativo de Navegação é responsável por orientar o usuário até a sala desejada utilizando Realidade Aumentada. Ele consome o arquivo JSON exportado pelo Coletor e, a partir da definição de origem e destino, calcula a rota no grafo e apresenta instruções passo a passo diretamente sobre o ambiente real. A navegação utiliza leitura de QR Code para identificar a posição inicial, exibe apenas uma esfera por vez para evitar poluição visual e apresenta mensagens de direção com base nas informações registradas no JSON da rota.

O cadastro e a autenticação do usuário não são realizados diretamente pelo aplicativo. Os dados utilizados no login, como matrícula e e-mail, são previamente definidos no processo de inscrição da Interação FURB, realizado por meio do sistema web oficial do evento. Nesse sistema, o participante efetua sua inscrição, informa seus dados pessoais e seleciona as oficinas de interesse, podendo posteriormente visualizar e imprimir o comprovante de inscrição. O aplicativo assume que essas informações já estão disponíveis e utiliza a matrícula e o e-mail apenas como forma de identificação do usuário e de acesso às oficinas previamente vinculadas, não sendo implementadas funcionalidades de CRUD ou integração direta com banco de dados institucional da FURB.

Para o aplicativo de Navegação, foram definidos os seguintes casos de uso principais: autenticar usuário, visualizar oficinas vinculadas, selecionar oficina, definir origem da rota, iniciar navegação em Realidade Aumentada e acompanhar o trajeto até o destino. Esses casos de uso descrevem as interações essenciais entre o usuário visitante e o sistema, servindo de base para os requisitos funcionais apresentados a seguir.

A seguir são apresentados os RFs (Quadro 6) e RNFs (Quadro 7) definidos para o aplicativo de Navegação.

Quadro 6 – RFs do aplicativo de `Navegação`

RF01	O aplicativo deve permitir que o usuário realize login informando matrícula e e-mail.
RF02	O aplicativo deve permitir exibir as oficinas vinculadas ao usuário.
RF03	O aplicativo deve permitir visualizar detalhes de cada oficina.
RF04	O aplicativo deve permitir abrir a visualização em RA do painel 3D.
RF05	O aplicativo deve permitir definir a origem manualmente ou por leitura de QR Code.
RF06	O aplicativo deve permitir carregar automaticamente o destino com base na oficina selecionada.
RF07	O aplicativo deve permitir ler QR Codes utilizando a biblioteca ZXing.
RF08	O aplicativo deve permitir calcular a rota entre origem e destino utilizando o grafo.
RF09	O aplicativo deve permitir instanciar a esfera roxa correspondente ao primeiro ponto da rota.
RF10	O aplicativo deve permitir detectar a proximidade do usuário em relação à esfera exibida.
RF11	O aplicativo deve permitir ao alcançar a esfera, alterar sua cor para amarela e removê-la da cena.
RF12	O aplicativo deve permitir instanciar a próxima esfera roxa ao avançar para o próximo ponto.
RF13	O aplicativo deve permitir exibir mensagens na interface, baseadas no sentido da rota.
RF14	O aplicativo deve permitir exibir a esfera verde no destino.

Fonte: elaborado pelo autor, 2025.

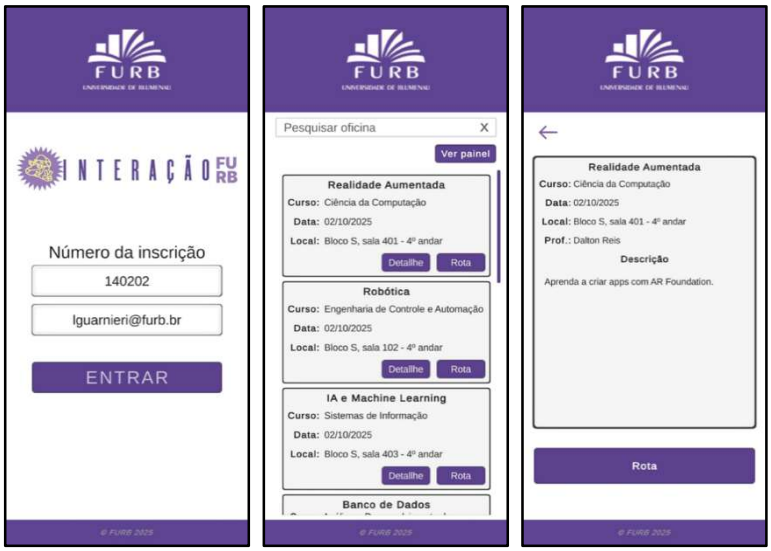
Quadro 7 – RNFs do aplicativo de `Navegação`

RNF01	O aplicativo deve funcionar mesmo sem conexão à internet após o carregamento dos dados da rota.
RNF02	O aplicativo deve ler o QR Code com baixa latência.
RNF03	O aplicativo deve manter a renderização e o tracking com desempenho mínimo de 30 FPS.
RNF04	O aplicativo deve ser compatível com dispositivos Android compatíveis com ARCore.
RNF05	O aplicativo deve efetuar a troca de esferas de forma fluida e sem atrasos perceptíveis.
RNF06	O aplicativo deve ter o design da interface simples, exibindo apenas elementos essenciais.
RNF07	O aplicativo deve ter a detecção de proximidade estável em diferentes condições de iluminação.
RNF08	O aplicativo deve ser intuitivo e rápido para o usuário entre a leitura do QR Code e início da rota.

Fonte: elaborado pelo autor, 2025.

O fluxo do aplicativo de Navegação inicia com a autenticação do usuário. Ao abrir o aplicativo (Figura 3a), o usuário é direcionado à tela de login, onde deve informar matrícula e e-mail seguindo o padrão usado na Interação FURB. Esse controle garante que apenas participantes inscritos tenham acesso às oficinas e às rotas associadas. Após o login, o aplicativo de Navegação carrega as oficinas vinculadas à matrícula informada. Cada oficina é exibida em formato de *card* contendo nome (da oficina), curso, data, local e as opções de Detalhe e ver rota (Figura 3b). Também está disponível o botão Ver Painel (mais acima), que exibe um painel informativo em Realidade Aumentada. Ao tocar em Detalhe, o usuário visualiza um *card* ampliado com informações adicionais sobre a oficina, mantendo também o botão para iniciar a Rota (Figura 3c).

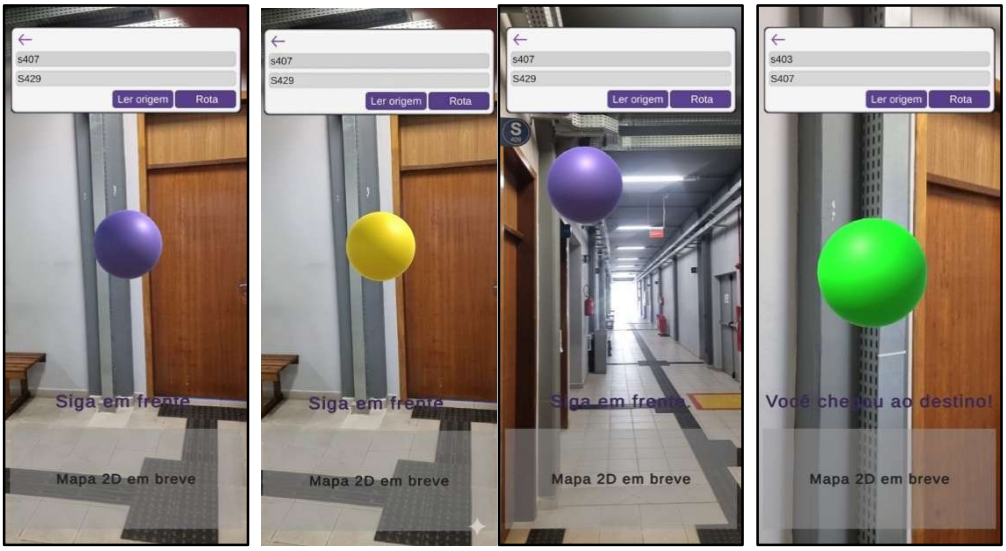
Figura 3 – Telas: (a) login do aplicativo; (b) oficinas; (c) detalhes



Fonte: elaborado pelo autor, 2025.

Dessa forma, no aplicativo de Navegação, o fluxo de navegação começa quando o usuário seleciona Rota. Nessa etapa, o aplicativo solicita a definição da origem (o ponto de partida da rota). O usuário pode digitar manualmente ou apontar a câmera para um QR Code fixado em algum lugar no Bloco S, permitindo identificar então a localização inicial (Figura a). Depois de definir origem, com o destino já definido pela oficina selecionada, o aplicativo de Navegação inicia a cena de RA. O AR Foundation é ativado, iniciando o rastreamento espacial para ancorar corretamente os objetos virtuais. A primeira esfera roxa é exibida no ambiente real, representando o primeiro ponto da rota. Durante a navegação, o aplicativo de Navegação detecta a proximidade do usuário em relação à esfera. Quando essa aproximação ocorre, a esfera muda para amarela, indicando que o ponto foi alcançado (Figura b). Em seguida, ela é removida da cena e a próxima esfera roxa é instanciada (Figura c), guiando o usuário sequencialmente até o destino. Além das esferas, o aplicativo exibe instruções textuais na parte inferior da tela, com mensagens como vire à direita, vire à esquerda ou siga em frente, reforçando o sentido da próxima movimentação. Ao chegar ao último ponto, a esfera exibida é apresentada na cor verde, indicando que o destino foi alcançado (Figura d).

Figura 4 – Telas da rota: (a) esfera inicial; (b) esfera alcançada; (c) próxima esfera; (d) esfera final

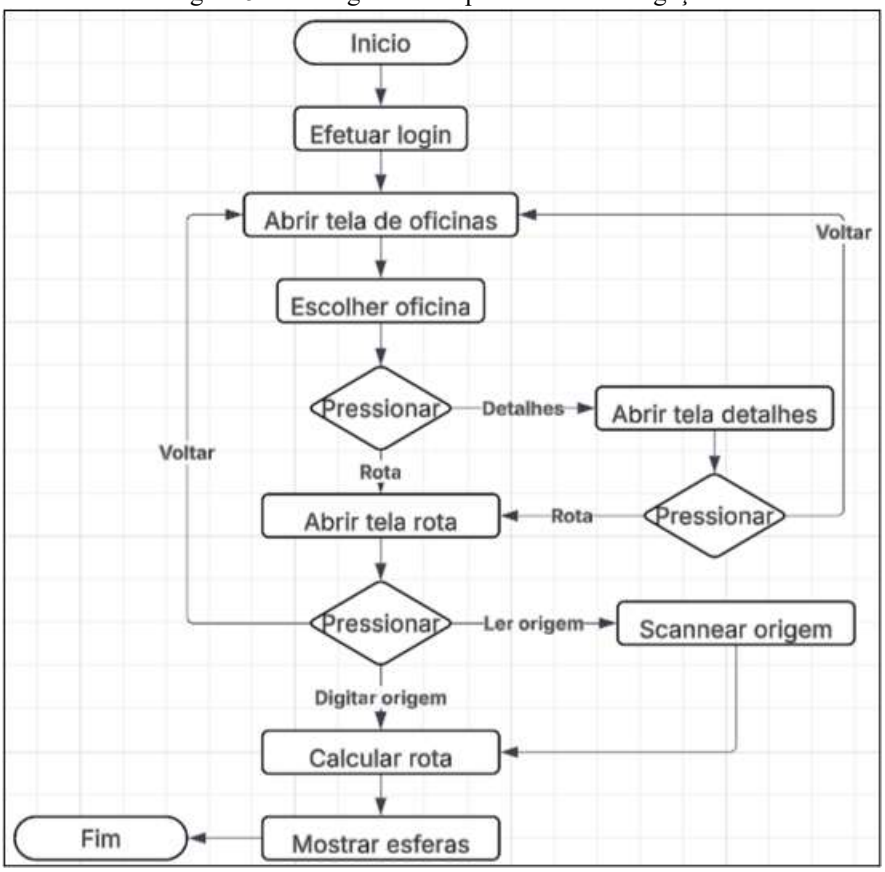


Fonte: elaborado pelo autor, 2025.

Esse fluxo progressivo, baseado em uma única esfera por vez, reduz distrações e facilita o entendimento da rota. A combinação de cores, proximidade e mensagens textuais torna a navegação clara e acessível mesmo para visitantes que não têm familiaridade com o Bloco S da FURB.

Com o objetivo de sintetizar visualmente as etapas descritas anteriormente, desde a autenticação do usuário até a conclusão da navegação em Realidade Aumentada, o fluxograma (Figura 5) apresentado a seguir resume o fluxo de funcionamento do aplicativo de Navegação, destacando as principais decisões e ações realizadas pelo sistema ao longo do processo.

Figura 5 – Fluxograma do aplicativo de Navegação



Fonte: elaborado pelo autor, 2025.

3.2 IMPLEMENTAÇÃO

A implementação dos dois aplicativos (Coletor e Navegação) foi realizada no motor Unity, utilizando a versão 6000.0.29f1, em conjunto com o AR Foundation 6.0.6 e o Google ARCore XR Plugin 6.0.6. Esses componentes são responsáveis pelo rastreamento, detecção do ambiente e suporte à renderização dos objetos virtuais nos dispositivos Android. Todo o desenvolvimento foi escrito em C-Sharp, utilizando o Visual Studio 2022 como ambiente de edição. Também foram empregados módulos do Unity essenciais ao ecossistema de Realidade Estendida (Extended Reality – XR), como XR Plugin Management, XR Core Utilities e o Input System, necessários para gerenciar dispositivos, câmeras e eventos de toque durante o uso dos aplicativos.

Os testes práticos foram executados em dois dispositivos Android, um Motorola Moto G7 (Android 10) e um Samsung S21 (Android 15). Essa escolha permitiu validar o comportamento dos aplicativos em aparelhos com capacidades diferentes de processamento, garantindo uma avaliação mais realista do desempenho, da estabilidade do *tracking* e da compatibilidade geral da aplicação. A seguir, são apresentados os principais aspectos técnicos da implementação de cada um dos aplicativos. No aplicativo Coletor, são detalhados o mecanismo de captura da posição espacial, a estrutura interna dos *prefabs* utilizados e o processo de exportação dos pontos para JSON. Já no aplicativo de Navegação, são descritos o processamento da leitura de QR Code, o cálculo da rota no grafo, o gerenciamento das esferas exibidas em RA e a lógica de detecção de proximidade que guia o usuário até o destino.

3.2.1 Aplicativo Coletor

O aplicativo Coletor é responsável por registrar fisicamente os pontos do Bloco S e gerar o arquivo pontos JSON, consumido posteriormente pelo aplicativo de Navegação. Sua implementação envolve a criação das informações descritivas de cada ponto, a captura da posição espacial fornecida pela AR Camera, a definição de uma origem comum para o aplicativo de coordenadas e a exportação dos dados no formato estruturado para reconstrução no aplicativo principal.

O fluxo começa com o preenchimento dos dados básicos do ponto, como bloco, tipo, andar e número da sala quando aplicável (Quadro 8). A classe ColetorController é responsável por gerar o identificador único (ID) de cada ponto. Esse ID segue um padrão que diferencia salas, corredores, escadas e elevadores, garantindo consistência na navegação posterior. Para pontos do tipo sala, o ID utiliza bloco e número. Nos demais casos, é adotado um prefixo específico, seguido do bloco, andar e um contador incremental.

Quadro 8 –Função responsável pela geração padronizada de IDs no Coletor

```
public void AtualizarID()
{
    string bloco = inputBloc.text.Trim().ToUpper();
    inputBloc.text = bloco;
    string tipo = dropdownTipo.options[dropdownTipo.value].text.ToUpper();
    int andar = dropdownAndar.value + 1;
    string id;
    if (tipo == "SALA") { id = $"{bloco}{inputNumeroSala.text}"; }
    else
    {
        string prefixo = tipo == "CORREDOR" ? "C" : (tipo == "ESCADA" ? "ES" : "EL");
        string chave = $"{prefixo}_{bloco}_{andar}";
        id = $"{chave}_{ProximoNumero(chave)}";
    }
    labelID.text = "ID: " + id;
}
```

Fonte: elaborado pelo autor, 2025.

Esse mecanismo (Quadro 8) garante que todos os pontos sigam a mesma convenção, o que facilita a leitura do JSON e a reconstrução do grafo no aplicativo de Navegação. Após preencher os dados e tocar na tela, o usuário gera um preview da posição atual. O aplicativo Coletor utiliza diretamente a posição absoluta da AR Camera para instanciar a esfera temporária. Esse preview permite verificar se o ponto está corretamente alinhado com o ambiente antes da confirmação. Ao confirmar o ponto com o botão Salvar Ponto, o aplicativo registra essa posição. A primeira posição salva define a origem do aplicativo de coordenadas, servindo como referência para todas as demais (Quadro 9). A partir do segundo ponto, todas as posições são convertidas para coordenadas relativas, removendo variações de rotação capturadas no momento da coleta. Isso garante que a reconstrução no aplicativo de navegação não seja distorcida por mudanças no sentido da câmera durante a coleta. Esse processamento é fundamental para manter coerência espacial quando o aplicativo de Navegação reconstruir o grafo em RA.

Quadro 9 – Trecho que converte posições absolutas em posições relativas e define a origem do aplicativo

```
Vector3 posAbs = esferaPreview.transform.position;
Ponto novo;
if (!origemDefinida)
{
    origemPos = posAbs;
    origemRot = Camera.main.transform.rotation;
    origemDefinida = true;
    novo = new Ponto
    {
        id = id, bloco = bloco, tipo = tipo,
        andar = andar, descricao = descricao, pos = Vector3.zero
    };
}
else
{
    // apenas deslocamento relativo, sem aplicar rotação da câmera
    Vector3 posRel = posAbs - origemPos;
    posRel = Quaternion.Inverse(origemRot) * posRel;
    posRel.y = 0f;

    novo = new Ponto
    {
        id = id, bloco = bloco, tipo = tipo, andar = andar, descricao = descricao, pos = posRel
    };
}
pontos.Add(novo);
```

Fonte: elaborado pelo autor, 2025.

O aplicativo Coletor permite criar conexões entre pontos para formar o grafo utilizado no cálculo das rotas. Quando múltiplos pontos são selecionados e um novo é salvo, conexões recíprocas são criadas automaticamente, representando arestas no grafo. Para facilitar a depuração, o aplicativo Coletor desenha linhas visuais entre pontos conectados, permitindo validar a estrutura antes da exportação. Ao concluir o mapeamento, o usuário pode exportar o arquivo JSON contendo todos os pontos (Quadro 10), suas conexões e os parâmetros de origem espacial (posição e rotação). A classe responsável encapsula tudo em um PontoWrapper, garantindo que o aplicativo principal leia a estrutura completa. A combinação entre captura via RA, normalização das coordenadas e exportação estruturada garante que o aplicativo funcione de forma estável e coerente em ambientes internos, sem dependência de Sistema de Posicionamento Global (Global Positioning System - GPS) ou sensores adicionais.

Quadro 10 – Função responsável por criar e exportar o JSON

```
public void ExportarJSON()
 {
     if (pontos.Count == 0)
     {
         mensagemErro.text = "Nenhum ponto cadastrado.";
         return;
     }
     if (!origemDefinida)
     {
         mensagemErro.text = "Nenhum ponto de origem definido.";
         return;
     }
     var wrapper = new PontoWrapper
     {
         pontos = pontos, origemPos = origemPos, origemRot = origemRot
     };
     string json = JsonUtility.ToJson(wrapper, true);
     string path = Path.Combine(Application.persistentDataPath, "pontos.json");
     File.WriteAllText(path, json);
     mensagemErro.text = "Exportado em:\n" + path;
 }
```

Fonte: elaborado pelo autor, 2025.

3.2.2 Aplicativo de `Navegação`

O aplicativo de Navegação interpreta o conjunto de pontos coletados pelo aplicativo Coletor, reconstrói a estrutura espacial do Bloco S e orienta o usuário até o destino utilizando Realidade Aumentada (RA). Além de apenas exibir esferas no ambiente, ele é responsável por carregar os dados mapeados, transformar essas informações em uma estrutura de grafo, calcular a rota mais adequada entre origem e destino e, por fim, projetar visualmente o caminho na câmera do dispositivo. Dessa forma, o aplicativo atua como a camada que conecta os dados estáticos do arquivo gerado no aplicativo Coletor com a experiência dinâmica de navegação em tempo real.

A implementação do aplicativo de Navegação foi organizada em quatro etapas principais: carregamento do arquivo de pontos, construção do grafo, cálculo da menor rota e renderização progressiva das esferas diretamente no ambiente real. Cada uma dessas etapas depende da anterior, o que exige que o arquivo exportado (do aplicativo Coletor) esteja consistente e que a estrutura interna dos pontos seja corretamente interpretada. Qualquer falha nessa cadeia pode comprometer o cálculo das rotas ou a exibição correta dos marcadores em RA.

A navegação inicia com a leitura do arquivo pontos.json, realizada pela classe PontosLoader. Esse módulo acessa o arquivo armazenado na pasta StreamingAssets, lê o conteúdo em formato textual e desserializa a estrutura para um objeto PontosWrapper. Esse objeto reúne em memória todas as informações necessárias para que o aplicativo consiga reconstruir o ambiente mapeado, incluindo:

I. a lista completa de pontos cadastrados durante o uso do aplicativo Coletor;
II. as conexões definidas entre esses pontos, que representam as arestas do grafo;
III. a posição e a rotação de origem definidas no momento da coleta, utilizadas como referência espacial.

Com esses dados carregados, o construtor da classe Grafo percorre a lista de pontos e cria um nó para cada um deles, armazenando o identificador, a posição relativa e o conjunto de conexões associadas (Quadro 11). O resultado é uma estrutura em memória que representa o Bloco S como um grafo navegável, pronta para ser utilizada pelo algoritmo de cálculo de rotas e pela lógica de renderização das esferas em Realidade Aumentada.

Quadro 11 – Trecho da classe Grafo, responsável por criar cada nó com posição relativa e lista de conexões.

```
public Dictionary<string, No> nos = new Dictionary<string, No>();

// manter o contexto espacial do mapeamento
public Vector3 origemPos { get; private set; }
public Quaternion origemRot { get; private set; }

public Grafo(PontosWrapper wrapper)
{
    if (wrapper == null || wrapper.pontos == null)
    {
        Debug.LogError("[Grafo] Wrapper inválido ou sem pontos.");
        return;
    }

    origemPos = wrapper.origemPos;
    origemRot = wrapper.origemRot;

    foreach (var p in wrapper.pontos)
    {
        var pos = new Vector3(p.pos.x, p.pos.y, p.pos.z);
        var no = new No(p.id, pos, p.conexoes);
        nos[no.id] = no;
    }

    Debug.Log($"[Grafo] Grafo carregado com {nos.Count} nós.");
}
```

Fonte: elaborado pelo autor, 2025.

Após selecionar origem e destino, o aplicativo Navegação utiliza o método *CalcularRota*(), que aciona a implementação do algoritmo de Dijkstra. Esse processo é responsável por analisar todas as conexões existentes no grafo reconstruído e identificar qual é o caminho mais eficiente entre os dois pontos. O algoritmo percorre o grafo utilizando as posições relativas de cada nó, calculando a distância euclidiana entre os pontos conectados e acumulando o menor custo possível a cada iteração.

Durante a execução, o algoritmo de Dijkstra avalia cada nó como um possível candidato ao próximo passo da rota, comparando continuamente os valores de distância registrados no dicionário interno do algoritmo. Sempre que uma nova rota parcial apresenta um custo menor do que o anteriormente registrado, o algoritmo atualiza a referência desse caminho, garantindo que apenas a solução com menor distância permaneça registrada. Esse processo continua até que o destino seja alcançado ou que todos os nós conectados tenham sido explorados.

Ao final, o método (Quadro 12) retorna uma lista ordenada de identificadores representando o trajeto completo que o usuário deve percorrer. Essa lista já reflete a ordem correta dos pontos no ambiente, permitindo que o aplicativo instancie as esferas virtuais uma a uma no fluxo de navegação. Dessa forma, o cálculo da rota não apenas define o percurso ideal, mas também organiza toda a sequência de visualização utilizada pelo aplicativo durante a projeção das esferas em Realidade Aumentada.

Quadro 12 – Algoritmo de Dijkstra para calcular o menor caminho entre os nós

```
var dist = new Dictionary<string, float>();
var anterior = new Dictionary<string, string>();
var naoVisitados = new List<string>();

foreach (var id in grafo.nos.Keys)
{
    dist[id] = float.PositiveInfinity;
    naoVisitados.Add(id);
}
dist[origem] = 0;

while (naoVisitados.Count > 0)
{
    string atual = null;
    float menorDist = float.PositiveInfinity;

    foreach (var id in naoVisitados)
    {
        if (dist[id] < menorDist)
        {
            menorDist = dist[id];
            atual = id;
        }
    }

    if (atual == null) break;
    naoVisitados.Remove(atual);

    if (atual == destino) break;

    foreach (var vizinhoId in grafo.nos[atual].conexoes)
    {
        if (!grafo.nos.ContainsKey(vizinhoId))
        {
            Debug.LogWarning($"Conexão inválida: {atual} -> {vizinhoId}");
            continue;
        }

        float novaDist = dist[atual] + Vector3.Distance(
            grafo.nos[atual].pos,
            grafo.nos[vizinhoId].pos
        );

        if (novaDist < dist[vizinhoId])
        {
            dist[vizinhoId] = novaDist;
            anterior[vizinhoId] = atual;
        }
    }
}
```

Fonte: elaborado pelo autor, 2025.

Como o aplicativo Coletor armazena apenas posições relativas, o aplicativo precisa reconstruir essas coordenadas no ambiente real antes de iniciar a navegação. Essa etapa é realizada por meio de uma calibração espacial que alinha o grafo ao espaço físico onde o usuário se encontra. Primeiro, o aplicativo calcula a posição absoluta do ponto de origem aplicando a rotação e a posição de referência capturadas no aplicativo Coletor sobre a posição relativa do nó inicial (Quadro 13). Em seguida, essa posição é comparada com a posição atual da AR Camera, permitindo identificar o deslocamento necessário entre o ambiente mapeado e o ambiente real no momento da navegação. O resultado desse processo é um vetor de translação global, utilizado para ajustar todas as posições do grafo, garantindo que cada esfera seja projetada no local correto do espaço físico.

Quadro 13 – Reconstrução da posição absoluta aplicando origem, rotação e translação global

```
// calibra o deslocamento global
 Vector3 posUsuario = arCamera.transform.position;
 Vector3 posOrigemGrafo = origemPos + (origemRot * grafo.nos[origem].pos);
 translacaoGlobal = posUsuario - posOrigemGrafo;

 calibrado = true;
 indiceAtual = 0;

 LimparMarcadores();
 MostrarProximoPonto();
```

Fonte: elaborado pelo autor, 2025.

Após a calibração espacial, inicia-se a etapa de renderização da rota. Em vez de exibir simultaneamente todos os pontos do trajeto, o que geraria sobrecarga visual e dificultaria a interpretação do caminho, o aplicativo projeta apenas uma esfera por vez diretamente no ambiente real. Essa abordagem reduz ruído na tela e garante que o usuário foque apenas no ponto que deve alcançar em cada momento. Cada esfera é instanciada na posição absoluta reconstruída a partir dos dados do aplicativo Coletor e recebe um material específico conforme sua função na rota. A cor roxa representa pontos intermediários, amarela indica que o ponto atual foi alcançado e a cor verde marca o destino.

O avanço entre os pontos (as esferas) ocorre de forma automática, pois a cada quadro o aplicativo de Navegação compara a posição da AR Camera com a posição da esfera ativa. Quando a distância entre ambos atinge ou fica abaixo do raio de chegada configurado, o aplicativo entende que o usuário concluiu aquele trecho do caminho. Nesse momento a esfera ativa é removida da cena e a próxima esfera roxa é instanciada, repetindo o processo até que o usuário chegue ao último nó. No ponto final o aplicativo Navegação exibe a esfera verde e apresenta uma mensagem informando que o destino foi alcançado.

Além da orientação visual, o aplicativo de Navegação também fornece instruções textuais que complementam o entendimento do trajeto. O método ObterDirecaoTexto calcula o ângulo entre o vetor da câmera e o vetor que aponta para o próximo ponto, identificando se o usuário deve seguir em frente, virar à direita, virar à esquerda ou realizar uma mudança de direção mais brusca. Quando a rota envolve mudança de andar, o texto informa explicitamente a necessidade de subir ou descer.

Essa combinação entre elementos visuais e instruções textuais permite que a navegação seja progressiva, intuitiva e estável, mesmo em ambientes internos onde o GPS não funciona. A solução utiliza apenas as coordenadas relativas criadas no aplicativo Coletor, a calibração local e os recursos de rastreamento do AR Foundation, sem exigir qualquer tipo de infraestrutura adicional no prédio. Dessa forma, o aplicativo de Navegação oferece uma experiência de navegação clara, leve e eficiente, adequada para orientar visitantes que não conhecem a estrutura do Bloco S.

4 RESULTADOS

Nesta seção são apresentados os resultados obtidos a partir dos testes realizados com usuários e a análise comparativa da solução desenvolvida com os trabalhos correlatos discutidos na Fundamentação Teórica (seção 2). Os testes ocorreram em ambiente real, dentro do Bloco S da FURB, permitindo avaliar o comportamento do aplicativo de Navegação em condições próximas às de uso durante eventos acadêmicos. O foco da avaliação foi observar a clareza visual das esferas de navegação, a facilidade de seguir o trajeto projetado em Realidade Aumentada e a estabilidade geral do aplicativo durante o deslocamento pelos corredores.

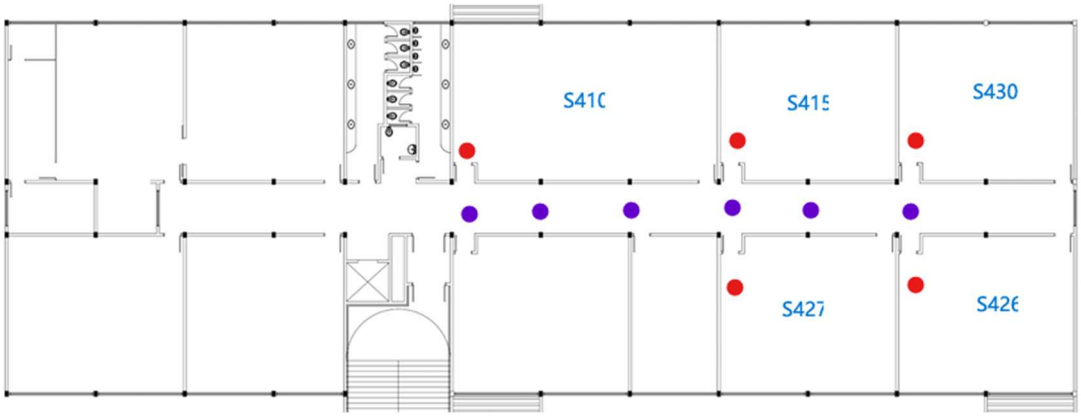
A análise também considera como as funcionalidades implementadas se posicionam em relação às soluções identificadas na literatura, observando pontos de aproximação, diferenças estruturais e limitações que influenciam diretamente o desempenho da navegação em ambientes internos. Essa combinação entre testes práticos e comparação teórica fornece uma visão completa do funcionamento do protótipo e permite identificar quais aspectos se mostraram eficazes e quais ainda podem ser aprimorados em versões futuras.

4.1 TESTES COM USUÁRIOS

Para avaliar a usabilidade do aplicativo, foram realizados testes com cinco participantes, três mulheres e dois homens, com idades entre 20 e 24 anos. Os participantes foram selecionados por conveniência, considerando a disponibilidade e o interesse em colaborar com o estudo, e apresentavam diferentes níveis de familiaridade com o Bloco S e com a Interação FURB. Antes do início dos testes, todos foram informados sobre os objetivos da avaliação e concordaram voluntariamente em participar. O aplicativo foi apresentado aos usuários com uma explicação breve sobre seu objetivo e funcionamento. Os testes foram realizados diretamente no ambiente real, utilizando a versão final do aplicativo de Navegação, com acompanhamento do pesquisador, permitindo que cada participante navegasse até um destino específico por meio das esferas exibidas em Realidade Aumentada.

Como forma de contextualizar os testes realizados e ilustrar de maneira mais concreta o percurso executado pelos participantes, a Figura 6 apresenta um exemplo de rota utilizada durante a avaliação do aplicativo de Navegação no Bloco S da FURB, especificamente no quarto andar. Na representação apresentada, os pontos em roxo correspondem aos pontos intermediários posicionados ao longo do corredor, enquanto os pontos em vermelho indicam as salas utilizadas como destino da navegação. A visualização da rota permite compreender melhor o cenário real no qual o aplicativo foi avaliado, bem como a disposição espacial dos pontos que serviram de referência para a navegação em Realidade Aumentada. Essa representação auxilia na interpretação dos resultados obtidos, especialmente no que se refere à precisão percebida da rota e à estabilidade das esferas durante o deslocamento dos usuários.

Figura 6 – Exemplo de rota utilizada nos testes do aplicativo de Navegação no quarto andar do Bloco S



Fonte: elaborado pelo autor, 2025.

A fim de caracterizar o grupo de usuários que participou dos testes, o Quadro 14 apresenta o perfil dos participantes, incluindo informações relacionadas à faixa etária, vínculo com a FURB, familiaridade com o Bloco S, participação prévia na Interação FURB e experiência anterior com aplicativos de navegação e Realidade Aumentada.

Quadro 14 – Perfil dos Usuários

Pergunta	Resposta	Quantidade
Idade (20–24 anos)	Sim	100%
Estuda atualmente na FURB	Sim	20%
Já visitou o Bloco S antes	Sim	80%
Já participou da Interação FURB	Sim	80%
Possui experiência prévia com RA	Sim	40%
Utiliza aplicativos de navegação (Maps/Waze)	Sim	100%

Fonte: elaborado pelo autor, 2025.

A análise do perfil indica que a maior parte dos participantes já possuía algum nível de familiaridade com o Bloco S e com o uso de aplicativos de navegação, o que contribuiu para a compreensão geral do percurso. Ao mesmo tempo, a presença de um participante sem experiência prévia mostrou que o aplicativo também pode ser avaliado por alguém que se comporta como um visitante do evento. Os resultados mostraram que todos os usuários conseguiram compreender o fluxo principal do aplicativo e iniciar a navegação sem grandes dificuldades. A leitura do QR Code apresentou desempenho eficiente desempenho e foi considerada rápida e precisa. A etapa de navegação, por outro lado, evidenciou as limitações esperadas de aplicativos de RA baseados exclusivamente em visão computacional, especialmente em corredores do Bloco S, onde variações no tracking causaram pequenos deslocamentos das esferas durante o movimento. O conjunto das avaliações relacionadas a experiência de navegação são apresentados no Quadro 15.

Quadro 15 – Usabilidade e experiência de navegação

Pergunta	Resposta	Quantidade
Conseguiu iniciar a rota sem dificuldade	Sim	80%
Conseguiu ler o QR Code corretamente	Sim	100%
As instruções em texto foram claras	Sim	80%
As esferas estavam visíveis e fáceis de identificar	Sim	60%
A rota exibida foi precisa o suficiente para chegar ao destino	Sim	60%
Conseguiu concluir o trajeto até a sala corretamente	Sim	80%
Considerou a navegação simples e intuitiva	Sim	80%

Fonte: elaborado pelo autor, 2025.

Os dados mostram que a maior parte dos participantes conseguiu concluir o trajeto corretamente, embora apenas seis em cada dez consideraram a rota suficientemente precisa para orientação completa. Esse comportamento está diretamente relacionado à instabilidade do *tracking*, apontada por todos como o principal problema. As esferas eram visíveis, porém apresentaram pequenas oscilações que exigiram maior atenção do usuário para confirmar a direção. Ainda assim, quatro dos cinco participantes avaliaram as instruções textuais como claras, indicando que essa camada complementar de orientação contribuiu para compensar parte das limitações do posicionamento virtual.

A percepção geral dos usuários foi moderada e refletiu tanto os acertos quanto os limites do protótipo. O design das telas foi considerado claro pela maioria e todos reconheceram que a navegação por esferas 3D é intuitiva. Ao mesmo tempo, a ausência de um mapa 2D para apoia a navegação, e a instabilidade natural da RA em movimento fizeram com

que a confiança no aplicativo permanecesse parcial para a maioria. A avaliação geral do aplicativo de Navegação está apresentada no Quadro 16.

Quadro 16 – Avaliação Geral do Aplicativo

Pergunta	Resposta	Quantidade
A experiência geral foi satisfatória	Sim	80%
O aplicativo ajudou a encontrar o destino corretamente	Parcial	40%
A navegação com esferas 3D foi intuitiva	Sim	100%
A instabilidade do AR prejudicou o uso	Sim	60%
A rota exibida esteve alinhada com o ambiente real	Parcial	60%
Você confiaria no aplicativo para se orientar sozinho	Parcial	80%
O design das telas foi claro e compreensível	Sim	80%
O aplicativo seria útil para visitantes	Sim	60%
O aplicativo precisa de melhorias significativas	Sim	100%

Fonte: elaborado pelo autor, 2025.

De forma geral, os usuários consideraram o aplicativo promissor e útil principalmente para visitantes, desde que sejam realizadas melhorias na estabilidade da RA e no alinhamento das esferas com o ambiente real. A necessidade de ajustes foi reconhecida por todos os participantes, reforçando que o protótipo cumpre sua finalidade inicial, mas ainda não está pronto para uso amplo durante eventos como a Interação FURB. Embora 80% dos usuários tenham concluído o trajeto corretamente, apenas 60% consideraram a rota suficientemente precisa, o que mostra uma diferença clara entre a execução prática da navegação e a percepção de confiança no aplicativo.

4.2 COMPARAÇÃO COM TRABALHOS CORRELATOS

A comparação com os trabalhos correlatos apresentados na Fundamentação Teórica permite situar a solução desenvolvida em relação ao estado atual das aplicações de navegação baseadas em Realidade Aumentada. Enquanto os estudos revisados apresentam estruturas tecnológicas mais robustas, o protótipo deste trabalho demonstra que é possível construir uma solução funcional mesmo com recursos mais simples e sem dependência de infraestrutura adicional.

Quadro 17 – Comparativo dos trabalhos correlatos

Características	Brasil Neto et al. (2023)	Rocha (2021)	Lu et al. (2021)	Este trabalho
Contexto / Aplicação	Turismo inteligente	Navegação em campus universitário (UFOP)	Navegação em campus universitário	Navegação indoor no Bloco S da FURB
Objetivo	Melhorar a experiência turística por meio de um aplicativo em Realidade Aumentada	Facilitar a navegação interna e externa no campus universitário	Aumentar a precisão e a experiência da navegação em campus universitário	Orientar visitantes em ambientes internos por meio de Realidade Aumentada
Tecnologia utilizada	ARCore e ARKit	ARKit (iOS)	ARCore	Unity, AR Foundation e ARCore
Estratégia de navegação	Exibição de informações aumentadas sobre locais turísticos	Navegação interna e externa baseada em RA	Integração de mapas 3D e sincronização contínua	Navegação ponto a ponto com esferas sequenciais
Principais funcionalidades	Painéis virtuais sobrepostos ao ambiente real	Marcadores e feedback visual em tempo real	Planejamento de rotas e visualização tridimensional	Renderização sequencial de esferas indicando o trajeto
Pontos fortes	Boa aceitação dos usuários e maior estabilidade dos painéis virtuais	Maior estabilidade e detecção espacial consistente	Alta precisão e integração entre navegação interna e externa	Solução simples, acessível e sem dependência de infraestrutura adicional
Pontos fracos / Limitações	Dependência de hardware específico e problemas de compatibilidade entre dispositivos	Restrição ao ecossistema Apple	Necessidade de infraestrutura digital mais robusta	Instabilidade do tracking e dependência da calibração inicial

Fonte: elaborado pelo autor, 2025.

O trabalho de Brasil Neto *et al*. (2023), voltado ao turismo inteligente, apresentou boa aceitação dos usuários e maior estabilidade na visualização dos painéis virtuais. Embora também utilize ARCore e ARKit, o foco desse aplicativo está na exibição de informações aumentadas e não na navegação ponto a ponto. A principal limitação relatada pelos autores foi a compatibilidade entre dispositivos, especialmente em aparelhos sem giroscópio ou bússola. No aplicativo Navegação, a restrição foi diferente: a maior dificuldade foi a instabilidade do *tracking* em ambiente interno, que afetou diretamente o alinhamento das esferas e a precisão da rota. Ou seja, enquanto o estudo de Brasil Neto *et al*. (2023) enfrentou barreiras de hardware, o presente projeto lidou com limitações próprias do rastreamento visual em corredores.

Rocha (2021) apresentou um aplicativo de navegação para a UFOP utilizando recursos nativos do ARKit, o que proporcionou maior estabilidade e detecção espacial mais consistente. Esse é um ponto crítico na comparação, já que o ecossistema Apple possui sensores mais precisos e integração nativa mais completa para RA, reduzindo flutuações nas âncoras virtuais e oferecendo resultados mais previsíveis. Naturalmente, isso gera uma diferença de maturidade entre os dois aplicativos. O aplicativo de Navegação, desenvolvido em Android com ARCore, apresenta maior variação na estabilidade, o que explica parte da diferença na precisão das rotas e na confiança dos usuários.

Já o aplicativo de Lu *et al*. (2021) se destaca pelo uso de mapas tridimensionais, sincronização contínua de posição e integração entre navegação interna e externa. A proposta combina RA com elementos de visualização mais complexos, além de utilizar técnicas de modelagem espacial que vão além do escopo deste TCC. Diferente disso, o aplicativo de Navegação utiliza um modelo simplificado baseado em esferas sequenciais e depende de uma única calibração inicial. Essa escolha torna o aplicativo mais acessível, porém mais sensível ao deslocamento relativo do usuário e às variações do *tracking*. A comparação demonstra que soluções mais completas exigem infraestrutura digital mais rica e sensores mais robustos do que aqueles disponíveis nas configurações utilizadas neste trabalho.

De forma geral, a análise mostra que o protótipo desenvolvido cumpre a proposta básica de orientar o usuário por meio de elementos de Realidade Aumentada, mas ainda não atinge o nível de estabilidade e precisão dos trabalhos correlatos. A principal diferença está na tecnologia empregada e na ausência de camadas adicionais de suporte, como mapas 3D, sincronização contínua ou sensores avançados. Ainda assim, o aplicativo demonstrou ser capaz de construir toda a cadeia necessária para navegação *indoor* em RA, desde o mapeamento dos pontos até a renderização progressiva da rota, utilizando apenas ferramentas acessíveis e sem exigir infraestrutura dedicada. As limitações observadas nos testes, especialmente a instabilidade do *tracking*, ajudam a explicar a diferença de desempenho em relação aos aplicativos mais maduros descritos na literatura.

5 CONCLUSÕES

O desenvolvimento dos aplicativos permitiu alcançar o objetivo principal deste trabalho, que foi criar uma solução de navegação indoor baseada em Realidade Aumentada capaz de orientar visitantes dentro do Bloco S da FURB. A fundamentação teórica apresentada forneceu suporte conceitual para a escolha da Realidade Aumentada como tecnologia central, bem como para a adoção de técnicas de navegação em ambientes internos, considerando as limitações do uso de GPS nesses contextos. Além disso, os estudos relacionados contribuíram para a definição da arquitetura da solução e para a compreensão dos desafios associados ao tracking e à orientação espacial em ambientes fechados.

A integração entre o aplicativo Coletor e o aplicativo de Navegação possibilitou registrar pontos do ambiente real, reconstruir essas informações no dispositivo do usuário e exibir o trajeto por meio de esferas virtuais em RA. Os testes realizados no próprio Bloco S mostraram que a abordagem com coordenadas relativas e calibração espacial funciona de maneira consistente, permitindo a navegação ponto a ponto sem dependência de GPS ou de infraestrutura adicional. As ferramentas utilizadas, especialmente Unity e AR Foundation, mostraram-se adequadas ao propósito do projeto, viabilizando a implementação de um protótipo funcional com recursos amplamente acessíveis.

Como principal contribuição, este trabalho demonstra a viabilidade do uso de Realidade Aumentada como alternativa prática para navegação indoor em ambientes acadêmicos, apresentando uma solução que pode ser aplicada em eventos institucionais, como a Interação FURB, e adaptada para outros espaços internos. A modelagem do grafo, o cálculo da menor rota e a renderização sequencial das esferas evidenciam que é possível construir um sistema de orientação eficiente mesmo sem o uso de mapas georreferenciados tradicionais.

Apesar dos resultados positivos, alguns desafios e limitações ficaram evidentes. A principal delas é a dependência da posição inicial do usuário, uma vez que a navegação precisa começar próxima ao ponto onde o mapeamento foi realizado, sob risco de desalinhamento das esferas. Além disso, a ausência de um mapa 2D auxiliar pode dificultar a compreensão espacial do ambiente, especialmente para usuários que não conhecem o prédio. Os painéis 3D previstos para complementar a experiência não foram finalizados, permanecendo em nível de protótipo. Outro desafio identificado refere-se à forma de armazenamento dos dados, baseada em arquivos CSV e em um único JSON gerado pelo aplicativo Coletor, que atende ao protótipo, mas não apresenta boa escalabilidade.

Como trabalhos futuros, propõe-se o aprimoramento da estabilidade do tracking, do alinhamento das esferas e da ancoragem dos pontos, bem como a implementação de um mapa 2D auxiliar, a finalização dos painéis 3D, a migração para um banco de dados centralizado e a expansão do mapeamento para outros blocos da universidade. Mesmo com as limitações identificadas, o projeto evidencia o potencial da Realidade Aumentada como uma abordagem inovadora para navegação indoor, oferecendo uma experiência diferenciada, moderna e com aplicabilidade real em contextos acadêmicos e institucionais.

6 REFERÊNCIAS

APPLE. ARKit: **Understanding World Tracking**. 2022. Disponível em: https://developer.apple.com/arkit. Acesso em: 23 out. 2025.

BRASIL NETO, L. et al. **Aplicativo móvel de realidade aumentada aplicado ao turismo integrado à plataforma GnomonGo.** Trabalho de Conclusão de Curso – Universidade Federal do Ceará, 2023.

CAVALCANTE, R.; SILVA, D. **Uso de realidade aumentada para apoio à navegação em ambientes internos.** Anais do Congresso da Sociedade Brasileira de Computação, 2021.

CAVALCANTE, R.; SILVA, D. **Aplicações móveis com Unity para apoio à orientação espacial em ambientes internos.** Anais do Simpósio Brasileiro de Sistemas de Informação, 2022.

COSTA, P.; AMARAL, L. **Aplicações de Realidade Aumentada com Unity para apoio ao aprendizado.** Anais do Congresso da Sociedade Brasileira de Computação, 2022.

FERREIRA, A.; GOMES, L. **QR Codes no apoio à aprendizagem móvel:** aplicação e resultados. Revista Novas Tecnologias na Educação, v. 18, n. 2, 2020.

GOOGLE. ARCore: **Geospatial & Motion Tracking Overview.** 2023. Disponível em: https://developers.google.com/ar. Acesso em: 12 nov. 2025.

LOPES, R.; TEICHRIEB, V. **Desenvolvimento de aplicações de realidade aumentada com Unity.** Anais do Congresso da Sociedade Brasileira de Computação, 2020.

LU, X. et al. **An ARCore-Based Augmented Reality Campus Navigation System.** Shanghai University of Electric Power, 2021.

NASCIMENTO, D.; FILGUEIRAS, L**. Uso de Unity 3D como ferramenta de apoio ao ensino e prototipagem rápida.** Anais do Workshop de Informática na Educação – SBC, 2021.

ROCHA, L. R. **Aplicativo de navegação interna e externa utilizando realidade aumentada na plataforma iOS.** Trabalho de Conclusão de Curso – Universidade Federal de Ouro Preto, 2021.

SANTOS, M.; PEREIRA, R. **Uso de Unity 3D para criação de ambientes virtuais de aprendizagem.** Revista Novas Tecnologias na Educação, 2021.

SINGH, M.; BANO, M. **A comprehensive review of augmented reality for indoor navigation.** Journal of Ambient Intelligence and Humanized Computing, v. 12, p. 10489–10512, 2021.

SOUZA, R.; LIMA, G. **Uso de QR Codes como apoio à navegação e identificação de ambientes em aplicações móveis.** Anais do Simpósio Brasileiro de Informática na Educação, 2021.

UNITY TECHNOLOGIES**. AR Foundation Overview.** 2023. Disponível em: https://docs.unity3d.com/Packages/com.unity.xr.arfoundation. Acesso em: 10 out. 2025.

UNITY TECHNOLOGIES. **Unity Manual:** Introduction to Unity. 2023. Disponível em: https://docs.unity3d.com. Acesso em: 15 out. 2025.

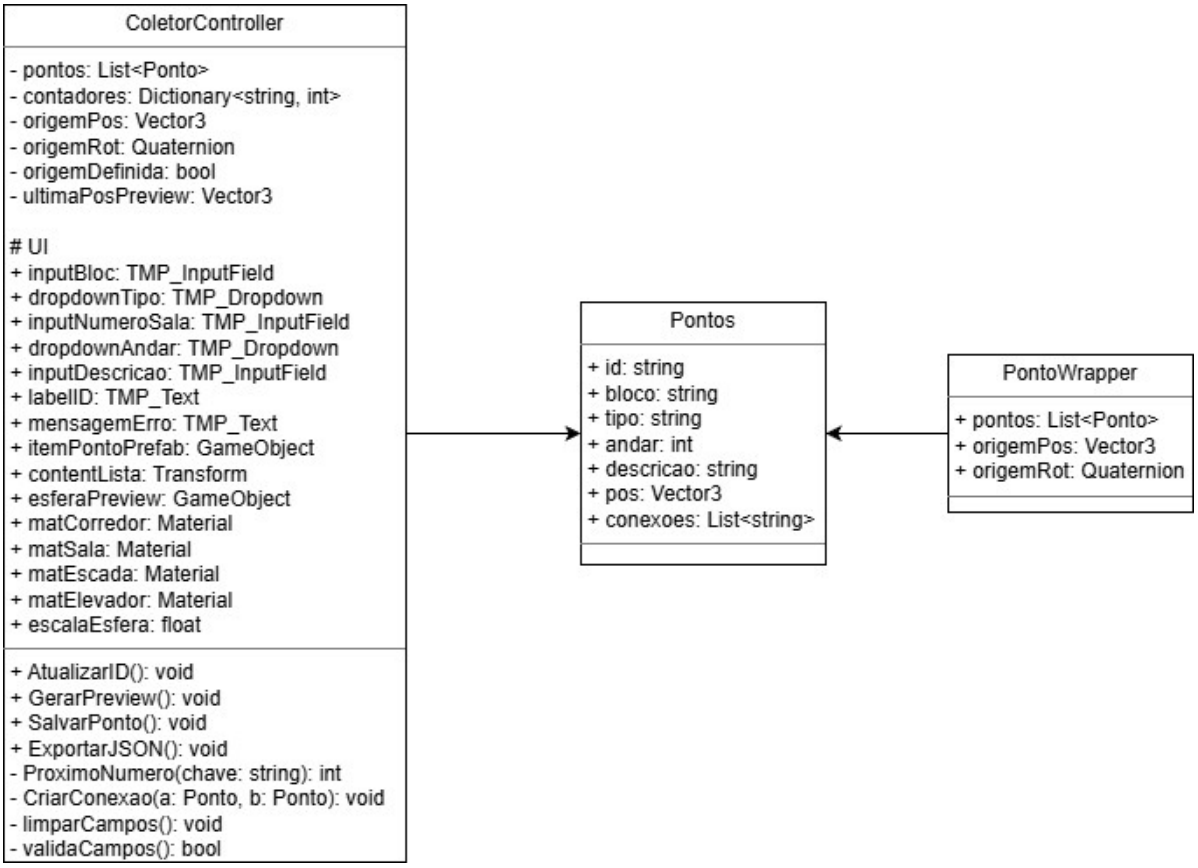
ZXING PROJECT**. ZXing: multi-format 1D/2D barcode image processing library.** 2023. Disponível em: https://github.com/zxing/zxing. Acesso em: 10 nov. 2025.

APÊNDICE A – DIAGRAMA DE CLASSES DO APLICATIVO COLETOR

A Figura 7 apresenta o diagrama de classes do aplicativo Coletor, responsável pelo processo de criação e registro de pontos de interesse em ambientes internos do Bloco S. O diagrama descreve as principais classes envolvidas no mapeamento dos ambientes, bem como seus atributos e relacionamentos, evidenciando como as informações descritivas (bloco, corredor, sala e descrição) e espaciais (coordenadas tridimensionais) são organizadas internamente pelo sistema.

Esse diagrama complementa a especificação apresentada no corpo do trabalho, permitindo compreender a estrutura lógica adotada para a captura, o armazenamento temporário dos pontos e a posterior exportação dos dados em formato JSON, que será consumido pelo aplicativo de Navegação.

Figura 7 – Diagrama de classes do aplicativo Coletor



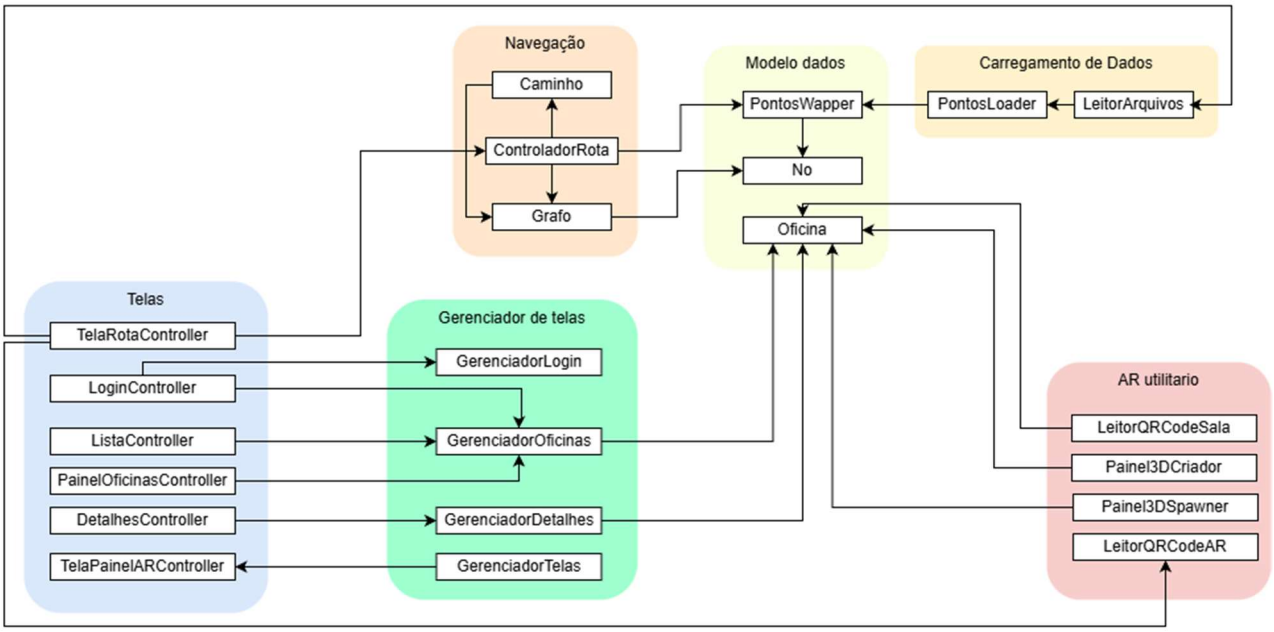
Fonte: elaborado pelo autor, 2025.

APÊNDICE B – ESTRUTURA DE CLASSES E MÓDULOS DO APLICATIVO DE NAVEGAÇÃO

A Figura 8 apresenta a estrutura de classes e módulos do aplicativo de Navegação. Diferentemente de um diagrama UML clássico, essa representação evidencia a organização interna do sistema, destacando os principais componentes, controladores e classes responsáveis pela leitura do arquivo JSON, autenticação do usuário, definição de origem e destino, cálculo de rotas no grafo e renderização das esferas em Realidade Aumentada.

O diagrama permite compreender como as responsabilidades estão distribuídas entre os módulos do aplicativo, bem como as dependências existentes entre as classes que gerenciam a navegação, o rastreamento espacial e a interação com o usuário. Essa estrutura complementa a especificação funcional apresentada no corpo do trabalho, fornecendo uma visão clara da arquitetura lógica adotada para o aplicativo de Navegação.

Figura 8 – Estrutura de classes e módulos do aplicativo de Navegação



Fonte: elaborado pelo autor, 2025.